

CO3 AT1 ASSESSMENTS

NAME: SIDDHI SRIVASTAVA. P

REG.NO:192521454

COURSE CODE: CSA0503

COURSE NAME: DATABASE MANAGEMENT SYSTEMS

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Sol:

Step 1: Understand the attributes

Attribute	Meaning
-----------	---------

SID	Student ID
-----	------------

SName	Student Name
-------	--------------

CID	Course ID
-----	-----------

CName	Course Name
-------	-------------

Instructor	Course Instructor
------------	-------------------

Grade	Student's Grade
-------	-----------------

A student can take many courses, and a course can have many students.

Therefore, one student-course combination is uniquely identified by:

(SID, CID)

Step 2: Identify Functional Dependencies

FD 1: SID → SName

A particular student ID identifies exactly one student name.

Therefore: **SID** $\rightarrow$ **SName**

FD 2: CID $\rightarrow$ **CName**

A particular course ID identifies exactly one course name.

Therefore: **CID** $\rightarrow$ **CName**

FD 3: CID $\rightarrow$ **Instructor**

Assuming each course has one instructor: **CID** $\rightarrow$ **Instructor**

FD 4: (SID, CID) $\rightarrow$ **Grade**

A grade depends on both the student and the course.

For example:

SID CID Grade

S01 C01 A

S01 C02 B

S02 C01 A+

The grade cannot be determined using only SID or only CID.

Therefore: **(SID, CID)** $\rightarrow$ **Grade**

Step 3: List the FDs

The main functional dependencies are:

SID $\rightarrow$ **SName**

CID $\rightarrow$ **CName**

CID $\rightarrow$ **Instructor**

SID, CID $\rightarrow$ **Grade**

Step 4: Find the Candidate Key

Try: SID

Starting with SID: **SID** $\rightarrow$ **SName**

So: **SID** = {**SID, SName**}

We don't get CID, CName, Instructor or Grade. Therefore, **SID is not a key**.

CID gives **CID → CName** and **CID → Instructor**

So: **CID** = {**CID, CName, Instructor**}

We don't get SID or Grade. Therefore, **CID is not a key**.

(SID, CID)

Start with: {**SID, CID**}

Using **SID → SName**: {**SID, CID, SName**}

Using **CID → CName**: {**SID, CID, SName, CName**}

Using **CID → Instructor**: {**SID, CID, SName, CName, Instructor**}

Using **(SID, CID) → Grade**: {**SID, CID, SName, CName, Instructor, Grade**}

We obtained **all attributes**.

Therefore: Candidate Key = **(SID, CID)**

Since neither SID nor CID alone can determine all attributes, the combination is minimal.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Sol: Suppose we have: **STUDENT(SID, SName, Courses)**

SID	SName	Courses
-----	-------	---------

101	Rahul	DBMS, OS, CN
-----	-------	--------------

102	Priya	AI, DBMS
-----	-------	----------

The **Courses** attribute contains multiple values. For example: **DBMS, OS, CN**. This violates **1NF**.

Step 1: Identify the repeating group

The **Courses** column contains multiple values: For Rahul: **DBMS, OS, CN**

Therefore, the value is not atomic.

Step 2: Convert values into atomic values

Instead of putting multiple courses into one row, create separate rows.

Before 1NF

SID	SName	Courses
101	Rahul	DBMS, OS, CN
102	Priya	AI, DBMS

After 1NF

SID	SName	Course
101	Rahul	DBMS
101	Rahul	OS
101	Rahul	CN
102	Priya	AI
102	Priya	DBMS

Now every cell contains **only one value**.

Step 3: Check 1NF conditions

A relation is in **1NF** when:

1. Every attribute contains atomic values.
2. There are no repeating groups.
3. Each row can be uniquely identified.

Here:

SID contains one value.
SName contains one value.
Course contains one value.

Therefore, the relation satisfies 1NF.

UNF: STUDENT(SID, SName, {Course})

where {Course} represents a repeating group.

1NF: STUDENT(SID, SName, Course)

with rows:

SID	SName	Course
101	Rahul	DBMS
101	Rahul	OS
101	Rahul	CN
102	Priya	AI
102	Priya	DBMS

Thus, the unnormalized relation has been converted into 1NF by removing repeating groups and making every attribute value atomic.

3. Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Sol:

Step 1: Find the candidate key

Calculate the closure of A.

A⁺

Initially: $A^+ = \{A\}$

Using: $A \rightarrow BC$

we get: $A^+ = \{A, B, C\}$

Using: $BC \rightarrow D$

we get: $A^+ = \{A, B, C, D\}$

Using: $D \rightarrow E$

we get: $A^+ = \{A, B, C, D, E\}$

Therefore: $A^+ = \{A, B, C, D, E\}$

So: Candidate Key = A

Step 2: Check for partial dependency

A relation is in 2NF if:

It is already in 1NF.

No non-prime attribute is partially dependent on a part of a composite candidate key.

Here, the candidate key is: **A**

The key contains only **one attribute**.

Therefore, a partial dependency is impossible.

For example: $(A,B) \rightarrow C$ and $A \rightarrow C$

Here C depends on only part of the key (A,B). But our key is simply A.

Therefore: **There are NO partial dependencies.**

Is decomposition required for 2NF - No.

The relation is already in **2NF**.

However, notice:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

There are transitive dependencies: $A \rightarrow BC \rightarrow D \rightarrow E$

Therefore, the relation is **not in 3NF**, even though it is in 2NF.

If decomposition into 3NF is required

We can decompose as: **R1(A, B, C)**

FD: **$A \rightarrow BC$**

Candidate key: **A**

R2(B, C, D)

FD: **$BC \rightarrow D$**

Candidate key: **BC**

R3(D, E)

FD: **$D \rightarrow E$**

Candidate key: **D**

Final decomposition:

R1(A,B,C)

R2(B,C,D)

R3(D,E)

R(A,B,C,D,E) is already in 2NF because its candidate key is A, which is a single attribute. There are **no partial dependencies** to eliminate.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Sol:

Step 1: Find the candidate key

EmpID uniquely identifies an employee.

Therefore: **EmpID** $\rightarrow$ **EmpName, DeptID**

From: **DeptID** $\rightarrow$ **DeptName, ManagerID**

we get: **EmpID** $\rightarrow$ **DeptName, ManagerID**

Therefore: **EmpID**⁺ = {**EmpID, EmpName, DeptID, DeptName, ManagerID**}

So: **Candidate Key = EmpID**

Step 2: Identify the transitive dependency

We have: **EmpID** $\rightarrow$ **DeptID** , **DeptID** $\rightarrow$ **DeptName** , **DeptID** $\rightarrow$ **ManagerID**

Therefore:

EmpID $\rightarrow$ **DeptID** $\rightarrow$ **DeptName**

EmpID $\rightarrow$ **DeptID** $\rightarrow$ **ManagerID**

So: **DeptName and ManagerID are transitively dependent on EmpID.**

Step 3: Remove the transitive dependency

Create two relations.

Relation 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Functional dependencies: $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

Primary Key: **EmpID**

Relation 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Functional dependencies: $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Primary Key: **DeptID**

Step 4: Check 3NF

EMPLOYEE : EmpID is the candidate key.

$\text{EmpID} \rightarrow \text{EmpName}$

$\text{EmpID} \rightarrow \text{DeptID}$

The determinant is a key. Therefore, it satisfies 3NF.

DEPARTMENT : DeptID is the candidate key.

$\text{DeptID} \rightarrow \text{DeptName}$

$\text{DeptID} \rightarrow \text{ManagerID}$

Again, the determinant is a key. Therefore, it satisfies 3NF.

Final 3NF Decomposition:

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C, C \rightarrow D, D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Sol:

Step 1: Find candidate keys

We first need to determine the candidate keys.

Find AB+

Start with: $AB^+ = \{A, B\}$

Using: $AB \rightarrow C$

we get: $AB^+ = \{A, B, C\}$

Using: $C \rightarrow D$

we get: $AB^+ = \{A, B, C, D\}$. Therefore: **AB is a candidate key.**

Find BC+

Start: $BC^+ = \{B, C\}$

Using: $C \rightarrow D$

we get: $BC^+ = \{B, C, D\}$

Using: $D \rightarrow A$

we get: $BC^+ = \{A, B, C, D\}$. Therefore: **BC is also a candidate key.**

Candidate Keys: AB , BC

Prime attributes: A, B, C

Non-prime attribute: D

Step 2: BCNF condition

A relation is in **BCNF** if for every non-trivial FD: $X \rightarrow Y$

X must be a superkey.

Check $AB \rightarrow C$: AB is a candidate key.

Therefore: $AB \rightarrow C$ satisfies BCNF.

Check $C \rightarrow D$

Calculate C^+ : $C^+ = \{C\}$

Using $C \rightarrow D$: $C^+ = \{C, D\}$

Using $D \rightarrow A$: $C^+ = \{C, D, A\}$

B is missing.

Therefore: **C is NOT a superkey.**

So: $C \rightarrow D$ it violates BCNF.

Check $D \rightarrow A$

Calculate D^+ : $D^+ = \{D\}$

Using $D \rightarrow A$: $D^+ = \{D, A\}$ here B and C are missing.

Therefore: **D is NOT a superkey.**

So: $D \rightarrow A$ it also violates BCNF.

Step 3: Decompose using $C \rightarrow D$

Since: $C \rightarrow D$ it violates BCNF, decompose R into: $R_1(C, D)$ and $R_2(A, B, C)$

Step 4: Check $R_1(C, D)$

FD: $C \rightarrow D$ here C determines all attributes of R_1 .

Therefore: **C is a candidate key of R_1 .** Hence R_1 is in BCNF.

Step 5: Check $R_2(A, B, C)$

The relevant FD is: $AB \rightarrow C$ and AB determines all attributes of R_2 .

Therefore: **AB is a candidate key.** Hence R_2 is in BCNF.

Final BCNF Decomposition : $R_1(C, D)$, $R_2(A, B, C)$.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Sol:

A decomposition is **lossless** if joining the decomposed relations gives exactly the original relation.

That is: $R = R_1 \bowtie R_2$. So, No spurious tuples should be generated.

There are three attributes: A B C. So, For each relation, create a row.

Use:

aA for distinguished symbol of A

aB for distinguished symbol of B

aC for distinguished symbol of C

Different symbols such as xA for missing attributes

Row A B C

R1(A,B) aA aB xC

R2(B,C) xA aB aC

Given: $B \rightarrow A$. We look for rows having the same value for B.

Both rows have: B = aB

Therefore, because: $B \rightarrow A$ the value of A must be the same in both rows. So we replace xA with aA.

Updated Tableau :

Row A B C

R1 aA aB xC

R2 aA aB aC

According to the Chase algorithm: If any row becomes completely distinguished, the decomposition is lossless.

Since Row 2 is: aA aB aC , the decomposition is **lossless**.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Sol:

Suppose teacher T1 teaches: DBMS, Python , Operating Systems

and has hobbies: Music , Cricket, Drawing.

If we store everything in one relation, we may get, unnecessary combinations.

A multi-valued dependency (MVD) is written as: $X \twoheadrightarrow Y$

Here, a teacher can have multiple subjects independently.

Therefore: MVD 1

TID $\rightarrow\rightarrow$ Subject

Meaning: For a particular TID, there can be multiple independent Subject values.

Similarly: MVD 2

TID $\rightarrow\rightarrow$ Hobby

Meaning: For a particular TID, there can be multiple independent Hobby values.

The relation contains two independent multi-valued facts:

TID $\rightarrow\rightarrow$ Subject

TID $\rightarrow\rightarrow$ Hobby

Combining them produces unnecessary combinations. This causes redundancy.

Check 4NF : A relation is in Fourth Normal Form (4NF) if:

For every non-trivial MVD:

$X \rightarrow Y$, X must be a **superkey**.

Here: TID $\rightarrow\rightarrow$ Subject, But TID is not a superkey of:

TEACHER(TID, Subject, Hobby) because TID alone doesn't determine one specific subject and hobby combination. Therefore, the relation violates 4NF.

Separate the independent multi-valued attributes.

Relation 1 : TEACHER_SUBJECT(TID, Subject)

Example:

TID Subject

T1 DBMS

T1 Python

T1 OS

Relation 2 : TEACHER_HOBBY(TID, Hobby)

Example:

TID Hobby

T1 Music

T1 Cricket

T1 Drawing

So, Original relation: TEACHER(TID, Subject, Hobby)

Multi-valued dependencies:

$TID \twoheadrightarrow Subject$

$TID \twoheadrightarrow Hobby$

**4NF decomposition: TEACHER_SUBJECT(TID, Subject)
TEACHER_HOBBY(TID, Hobby).**

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Sol:

Consider the relation: **SUPPLY(Supplier, Part, Project)**

Suppose a supplier supplies a particular part to a particular project.

Example:

Supplier Part Project

S1 P1 J1

S1 P1 J2

S1 P2 J1

S1 P2 J2

Identify the Redundancy

Suppose: **S1 supplies P1 and P2 and S1 supplies projects J1 and J2**

Then combinations may be stored repeatedly:

S1 - P1 - J1

S1 - P1 - J2

S1 - P2 - J1

S1 - P2 - J2

If there are many suppliers, parts, and projects, the number of combinations can become very large.

We can decompose: Relation 1 : **SP(Supplier, Part)**

Supplier Part

S1 P1

S1 P2

Relation 2 : SJ(Supplier, Project)

Supplier Project

S1 J1

S1 J2

Relation 3 : PJ(Part, Project)

Part Project

P1 J1

P1 J2

P2 J1

P2 J2

The original relation can be obtained by joining: **SP SJ PJ**

Thus: **SUPPLY(Supplier, Part, Project)** is decomposed into:

SP(Supplier, Part), SJ(Supplier, Project), PJ(Part, Project)

Original: SUPPLY(Supplier, Part, Project)

5NF Decomposition: SP(Supplier, Part), SJ(Supplier, Project) , PJ(Part, Project).

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Sol:

Consider: **COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, Instructor)**

Example:

StudentID	StudentName	DeptID	DeptName	CourseID	CourseName	Instructor
S01	Ravi	D01	CSE	C01	DBMS	Kumar
S01	Ravi	D01	CSE	C02	Python	Priya
S02	Anu	D01	CSE	C01	DBMS	Kumar

There is considerable repetition.

We can identify: FD 1 : **StudentID** → **StudentName, DeptID**

A StudentID determines the student's name and department.

FD 2 : DeptID → **DeptName**

A department ID determines the department name.

FD 3 : CourseID → **CourseName, Instructor**

A course ID determines the course name and instructor.

Candidate Key : A student can take multiple courses.

Therefore: **(StudentID, CourseID)** is the candidate key.

A. Insertion Anomaly

Suppose a new course is created:

CourseID = C03

CourseName = AI

Instructor = Meena , But no student has enrolled yet.

In the poorly designed table, we may not be able to insert the course information without having a StudentID.

B. Deletion Anomaly

Suppose S02 is the only student enrolled in C01. If we delete S02's record, we may accidentally lose: C01 , DBMS , Kumar . So information about the course is also lost.

C. Update Anomaly

Suppose the instructor for C01 changes: Kumar $\rightarrow$ Raj

If C01 appears in 100 rows, all 100 rows must be updated. If one row is missed, inconsistent information occurs.

Convert to 1NF

The table already contains atomic values. Therefore, it is in **1NF**.

COLLEGE(StudentID, StudentName, DeptID, DeptName,
CourseID, CourseName, Instructor)

Candidate key: **(StudentID, CourseID)**

Convert to 2NF

2NF removes **partial dependencies**.

Our key is: **(StudentID, CourseID)**

But: StudentID $\rightarrow$ StudentName, DeptID

StudentName and DeptID depend only on part of the key.

Also: CourseID $\rightarrow$ CourseName, Instructor

CourseName and Instructor depend only on CourseID.

Therefore, partial dependencies exist.

Now examine: **STUDENT(StudentID, StudentName, DeptID)**

We have: StudentID $\rightarrow$ StudentName, DeptID

But: DeptID $\rightarrow$ DeptName

Therefore: $\text{StudentID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$. This is a **transitive dependency**.

STUDENT : STUDENT(StudentID, StudentName, DeptID)

Primary Key: StudentID

Foreign Key: DeptID

DEPARTMENT : DEPARTMENT(DeptID, DeptName)

Primary Key: DeptID

COURSE : COURSE(CourseID, CourseName, Instructor)

Primary Key: CourseID

ENROLLMENT : ENROLLMENT(StudentID, CourseID)

Primary Key: (StudentID, CourseID)

Both StudentID and CourseID are foreign keys.

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Sol: Consider: **R(A, B, C, D, E)**

Functional dependencies:

1. **$A \rightarrow BC$**
2. **$BC \rightarrow D$**
3. **$D \rightarrow E$**

Find A

Start with: **A = {A}**

FD: **$A \rightarrow BC$**

Add B and C: **A = {A, B, C}**

Now we have B and C, so apply: **$BC \rightarrow D$**

Add D: **A = {A, B, C, D}**

Now apply: **$D \rightarrow E$**

Add E: **A = {A, B, C, D, E}**

Therefore: $A^+ = \{A, B, C, D, E\}$. Since A determines every attribute: A is a candidate key.

Find B

Start: $B^+ = \{B\}$. There is no FD beginning with B alone.

Therefore: $B^+ = \{B\}$. B is not a key.

Find C

Start: $C^+ = \{C\}$. No FD can be applied.

Therefore: $C^+ = \{C\}$. C is not a key.

Find D

Start: $D^+ = \{D\}$

Using: $D \rightarrow E$

we get: $D^+ = \{D, E\}$. D is not a key.

Find E

Start: $E^+ = \{E\}$. No FD can be applied.

Therefore: $E^+ = \{E\}$. E is not a key.

So: $AB^+ = \{A, B, C, D, E\}$

Therefore AB is a **superkey**.

But it is **not a candidate key**, because A alone is already a key.

A candidate key must be **minimal**.

